Text-to-SQL Semantic Parsing: A Comparative Evaluation of Zero-Shot and Few-Shot Prompting

Caterina Vespa
Sapienza University of Rome
vespa.2007103@studenti.uniroma1.it

Abstract

This report addresses the Text-to-SQL task, which aims to translate a natural language question into an executable SQL query given a database schema. We compare a zero-shot baseline and a few-shot prompting strategy that includes five structurally diverse examples for the same open-weight code language model, Qwen2.5-Coder-7B-Instruct (4-bit quantized), on the Spider dataset. Our results yield an Execution Accuracy of 70.21% for the zero-shot approach and 70.60% for the few-shot approach. The second approach gives only a marginal improvement, which an exact McNemar's test shows is not statistically significant ($p = 0.782$). Error analysis indicates that the few-shot examples induce generation truncation and push the model toward longer, alias-heavy SQL that exceeds the fixed 64-token generation budget. We discuss this and the shared error patterns (multi-table joins and `GROUP BY/HAVING`) that persist in both settings.

1 Task description/Problem statement

Text-to-SQL is a semantic parsing task that automatically translates a user's natural language question into a structured Query Language (SQL) executable on a relational database [2]. Given a natural language utterance Q and a database schema S (composed of tables, columns, primary keys, and foreign keys), the goal is to generate a valid SQL query Y that accurately retrieves the desired information. The task is cross-domain when, as in Spider, train and evaluation questions come from databases the model has never seen, which forces the model to generalize to unseen schemas rather than memorize a fixed set of tables [4].

1.1 Examples

Given the *flight_2* database schema, an example of an input and expected output is:

- **Input Question:** "How many United Airlines flights go to City 'Aberdeen'?"
- **Input Schema:**

```
Table: AIRLINES
  - uid (number) [PRIMARY KEY]
  - Airline (text)
Table: AIRPORTS
  - AirportCode (text) [PRIMARY KEY]
  - City (text)
Table: FLIGHTS
  - Airline (number)
  - DestAirport (text)
Foreign Keys:
  - FLIGHTS.Airline -> AIRLINES.uid
  - FLIGHTS.DestAirport -> AIRPORTS.
      AirportCode
```

- **Expected Output:**

```
SELECT count(*) FROM FLIGHTS AS T1
    JOIN AIRPORTS AS T2 ON T1.
    DestAirport = T2.AirportCode
    JOIN AIRLINES AS T3 ON T3.uid =
    T1.Airline WHERE T2.City = '
    Aberdeen' AND T3.Airline = '
    United Airlines'
```

1.2 Real-world applications

Text-to-SQL systems are highly valuable for Business Intelligence (BI) and data analytics. They empower non-technical users to interrogate complex relational databases via conversational assistants or Natural Language Interfaces to Databases (NLIDB), providing access to data without requiring SQL expertise.

2 Related work

Historically, Text-to-SQL has been addressed using sequence-to-sequence models (RAT-SQL, encoder-decoder parsers with schema linking) that were trained end-to-end on the Spider training set.

Recently, Large Language Models (LLMs) have dominated the state-of-the-art and Rajkumar et al. (2022) [3] showed that Codex, without any fine-tuning, is already a strong zero-shot baseline on Spider, and that adding a handful of examples to the prompt can match or beat previously trained models – the same zero-shot/fewshot contrast we study here, but with a smaller, open-weight, code-specialized model instead of a proprietary one.

More recent work, like DIN-SQL (Pourreza and Rafiei, 2023) [1], decomposes the task instead of relying on a single prompt. It breaks generation into schema linking, query classification, SQL generation and self-correction sub-steps, improving few-shot GPT-4 performance by roughly 10 points and reaching 85.3% execution accuracy on the Spider test set [1]. Qin et al. (2022) [2] provides a survey of Text-to-SQL methods, from rule-based and sequence-to-sequence systems to LLM-based prompting.

3 Datasets and benchmarks

Spider [4] is a large-scale, complex, and cross-domain Text-to-SQL dataset with 200 databases spanning 138 domains, split so that no database appears in more than one of the train/dev/test partitions. We use the standard development set (1034 question-query pairs) together with the corresponding SQLite databases and `tables.json` schema file, needed to execute predicted queries. Other relevant benchmarks include WikiSQL (single-table, simpler queries) and the more recent BIRD and Spider 2.0, which focus on massive, noisy database contents.

4 Existing tools, libraries, papers with code

To implement Text-to-SQL pipelines, the Hugging Face ecosystem is the standard: `transformers` and `accelerate` for loading and running the model, `bitsandbytes` for 4-bit quantization, and `datasets` for accessing Spider. The official Spider repository (`taoyds/spider` on GitHub) provides the reference evaluation scripts and baselines. For practical deployment, existing high-level frameworks like LangChain and LlamaIndex also offer built-in Text-to-SQL retrieval tools.

5 State-of-the-art evaluation

Text-to-SQL systems are evaluated using two primary metrics:

- **Exact Set Match (EM):** Compares the generated SQL with the gold SQL component-by-component and reports results broken down into four hardness levels (easy, medium, hard, extra hard) based on how many such components a query contains [4]. However this penalizes correct queries that are simply written differently from the gold one.
- **Execution Accuracy (EX):** Executes both the generated and gold SQL queries on the actual database and compares the returned result sets-the metric we also adopt in Section 6. EX is widely preferred, but Zhong et al. (2020) [5] show that plain execution accuracy on a single database instance still has a non-trivial error rate because a wrong query can coincidentally return the same result as the gold one.
- **Test-Suite Accuracy:** Executes each query against several automatically generated database instances chosen to maximize code coverage. This is now the official metric on the Spider leaderboard.

Across all these metrics, systems are scored only on databases never seen during training so that the score reflects generalization to new schemas rather than memorization.

6 Comparative evaluation

- **Datasets:** We used the Spider development set, containing 1034 examples over multiple unseen databases.
- **Systems and baselines:** We utilized Qwen2.5-Coder-7B-Instruct loaded in 4-bit precision and a fixed generation budget of 64 new tokens.
  - **Baseline (Zero-Shot):** A prompt containing the system instructions, the serialized database schema (tables, types, PKs, FKs), and the question.
  - **Proposed (Few-Shot):** The same prompt increased with 5 static examples extracted from the training set. To ensure diversity, these 5 examples were specifically selected to cover diverse SQL structures (`WHERE`, `COUNT + WHERE`, `ORDER BY`, `GROUP BY + COUNT`, `JOIN`).

- **Measures and protocol:** The predictions were evaluated using Execution Accuracy (EX) via a local SQLite engine. We use plain execution accuracy rather than test-suite accuracy, since both prompting conditions are compared on the exact same databases and queries, which limits the false-positive risk discussed in Section 5. To rigorously compare the two models, we applied McNemar's Test on the correct/incorrect outcomes.

6.1 Results

The results on the 1034 validation examples are summarized in Table 1.

Method	Correct	Wrong	Accuracy
Zero-Shot (Base)	726	308	70.21%
Few-Shot	730	304	70.60%

Table 1: Execution Accuracy on the Spider dev set.

Zero-shot prompting scores 70.21% (726/1034); few-shot prompting scores 70.60% (730/1034), an improvement of +0.39%. To verify if this difference was statistically meaningful, we built a 2x2 contingency table (669 cases correct for both, 247 wrong for both, 61 fixed by few-shot, and 57 regressed by few-shot). The McNemar test on this table gives a $p = 0.782$, so the difference is not statistically significant.

6.2 Discussion

The error analysis highlights severe limitations in the model's handling of complex structures. Out of the 247 cases failed by both approaches, the `JOIN` operator was present in 59.11% of the gold queries, followed by `WHERE` (50.20%) and `GROUP BY` (41.30%).

Construct	Zero-shot	Few-shot
JOIN	56.31%	53.88%
WHERE	70.47%	69.04%
GROUP BY	51.26%	57.40%
ORDER BY	68.88%	71.78%
HAVING	54.43%	56.96%
COUNT	67.72%	66.29%

Table 2: Execution accuracy by SQL construct in the gold query.

The aggregate improvement from few-shot prompting is small and statistically indistinguishable from noise, and Table 2 shows no consistent pattern: few-shot helps for `WHERE`, `GROUP BY`, `ORDER BY` and `HAVING`, but hurts on `JOIN` and `COUNT`.

Interestingly, the Few-Shot approach corrected 61 zero-shot errors but introduced errors in 57 previously correct predictions. Manual inspection reveals that providing examples occasionally caused the model to hallucinate table structures from the few-shot examples or prematurely truncate the generation (e.g., ending queries abruptly with `HAVING` or incomplete `COUNT` statements). This suggests that static few-shot prompting without dynamic retrieval based on the input question's semantic similarity may introduce noise rather than helpful context.

7 Conclusions

In this project, we compared zero-shot and few-shot Text-to-SQL generation using Qwen2.5-Coder-7B. For this type of model, five hand-picked few-shot examples do not yield a reliable improvement over the baseline: the +0.39% is not statistically significant, and the benefit is offset by a generation-length side effect that truncates otherwise-correct queries.

Future work should explore Dynamic Few-Shot selection (using vector embeddings to retrieve similar training examples) and self-correction mechanism where the model interacts with the SQLite error trace to fix its own queries.

Use of GenAI tools

Generative AI tools (specifically Google Gemini and Claude) were utilized strictly to help format this report from my own notebook, results, and analysis, synthesize the text into the required template, and ensure the English phrasing was academically rigorous. The author retains full responsibility for all content submitted.

References

[1] Mohammadreza Pourreza and Davood Rafiei. Din-sql: Decomposed in-context learning of text-to-sql with self-correction, 2023. 2

[2] Bowen Qin, Binyuan Hui, Lihan Wang, Min Yang, Jinyang Li, Binhua Li, Ruiying Geng, Rongyu Cao, Jian Sun, Luo Si, Fei Huang, and Yongbin Li. A survey on text-to-sql parsing: Concepts, methods, and future directions, 2022. 1, 2

[3] Nitarshan Rajkumar, Raymond Li, and Dzmitry Bahdanau. Evaluating the text-to-sql capabilities of large language models, 2022. 2

[4] Tao Yu, Rui Zhang, Kai Yang, Michihiro Yasunaga, Dongxu Wang, Zifan Li, James Ma, Irene Li, Qingning Yao, Shanelle Roman, Zilin Zhang, and Dragomir Radev. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task, 2019. 1, 2

[5] Ruiqi Zhong, Tao Yu, and Dan Klein. Semantic evaluation for text-to-sql with distilled test suites, 2020. 2